

Practical: 2

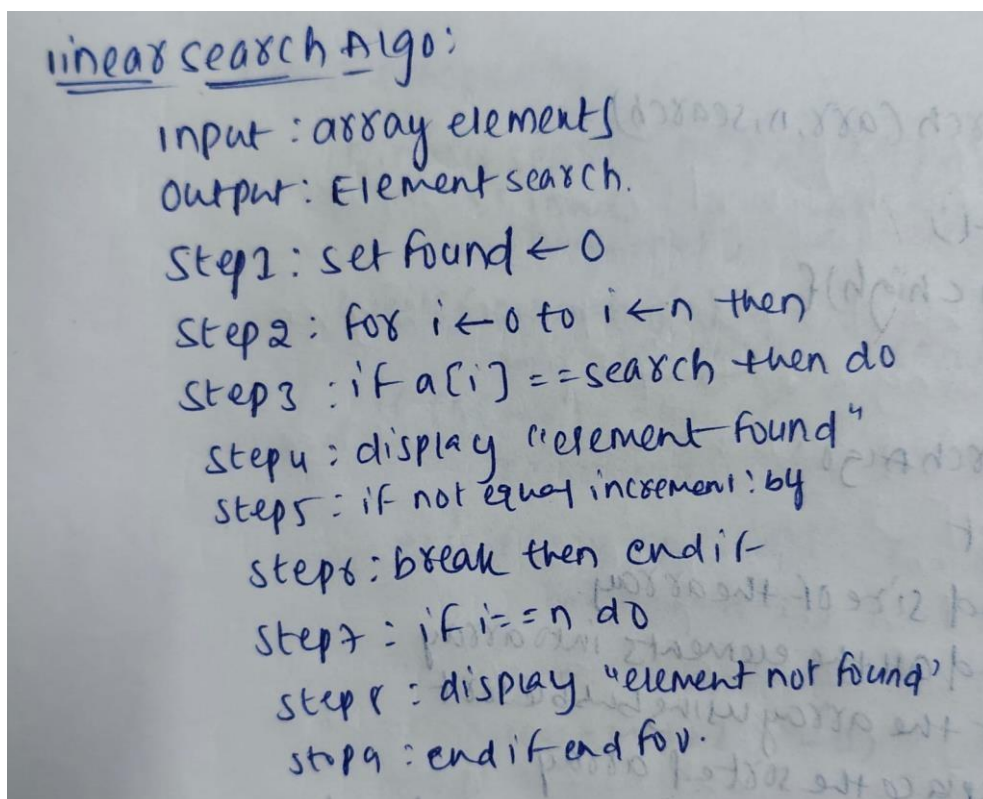
Implementation and Time analysis of linear and binary search algorithm

05/08/2026

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Linear search

Algorithm:



linear search Algo:
input : array element
output : Element search.
Step1 : set found $\leftarrow 0$
Step2 : for $i \leftarrow 0$ to $i \leftarrow n$ then
Step3 : if $a[i] == \text{search}$ then do
Step4 : display "element found"
Step5 : if not equal increment by
Step6 : break then end if
Step7 : if $i == n$ do
Step8 : display "element not found"
Step9 : end if end for.



Program:

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int target;
    cout << "Enter the element to search: ";
    cin >> target;

    int index = -1;
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            index = i;
            break;
        }
    }

    if (index != -1) {
        cout << "Element found at index: " << index << endl;
    } else {
        cout << "Element not found" << endl;
    }

    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements: 66
32
55
45
6
Enter the element to search: 6
Element found at index: 4
```

Time Complexity:

Best cases:-
element is found at the first search position.

```
for(i=0; i<n; i++){
    if(arr[i]==key){
        return i;
    }
}
```

$T(n) = 1$
 $T(n) = O(1)$

Average case:-

```
for(i=0; i<n; i++){
    if(arr[i]==key){
        return i;
    }
}
```

$T(n) = O(n)$

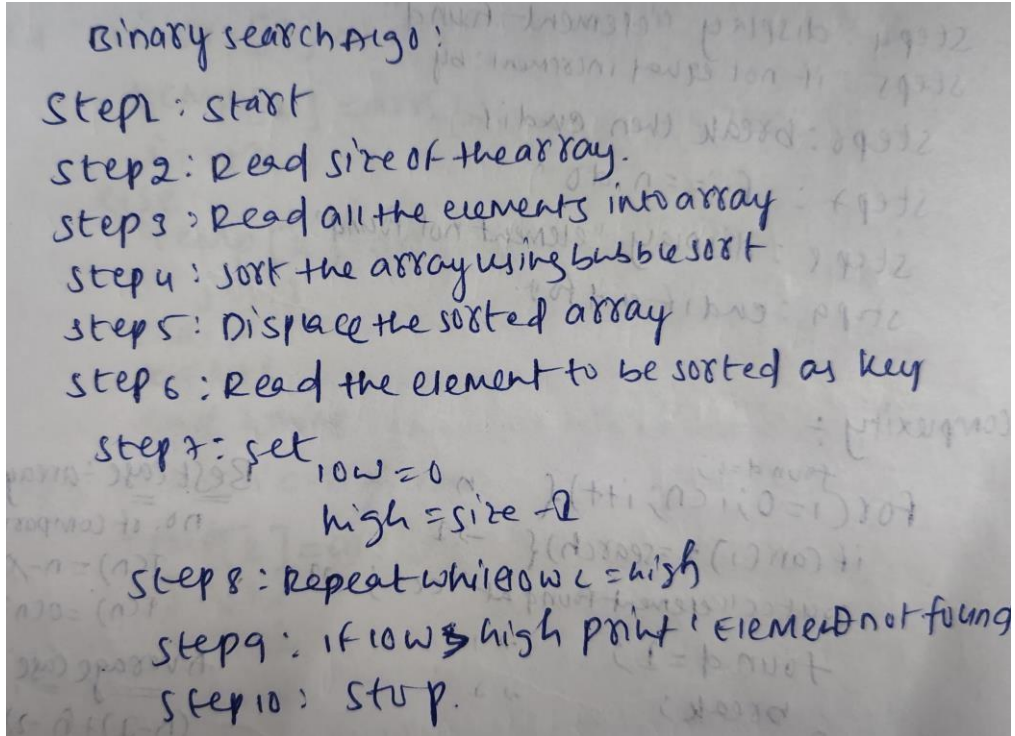
Worst case:-

```
for(i=0; i<n; i++){
    if(arr[i]==key){
        return i;
    }
}
```

$T(n) = O(n)$

Binary search

Algorithm:



Binary search Algo :

- Step 1: start
- Step 2: Read size of the array.
- Step 3: Read all the elements into array
- Step 4: Sort the array using bubble sort
- Step 5: Display the sorted array
- Step 6: Read the element to be sorted as key
- Step 7: set $low = 0$
 $high = size - 1$
- Step 8: Repeat while $low \leq high$
- Step 9: if $low > high$ print 'Element not found'
- Step 10: Stop.

Program code:

```
#include <iostream>
#include <vector>
using namespace std;
int binarySearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}
```

```
{
    int size;
    cout << "Enter the number of elements: ";
    cin >> size;

    vector<int> arr(size);
    cout << "Enter " << size << " elements in sorted order:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

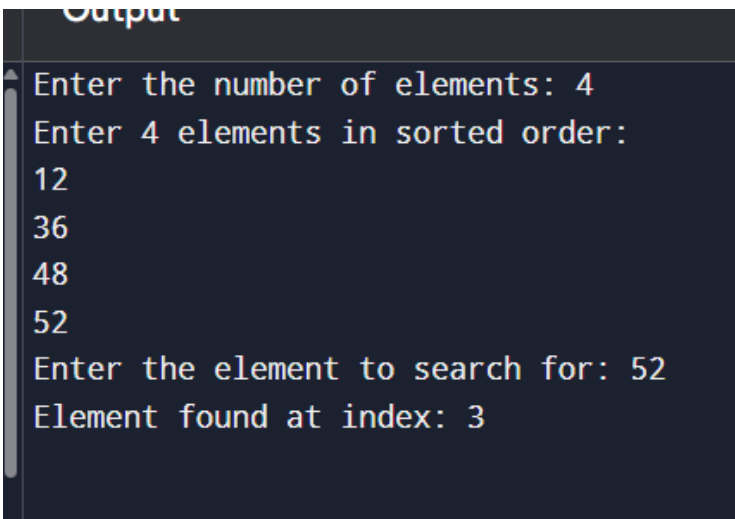
    int target;
    cout << "Enter the element to search for: ";
    cin >> target;

    int result = binarySearch(arr, target);

    if (result != -1) {
        cout << "Element found at index: " << result << endl;
    } else {
        cout << "Element not found in the array." << endl;
    }

    return 0;
}
```

Output:

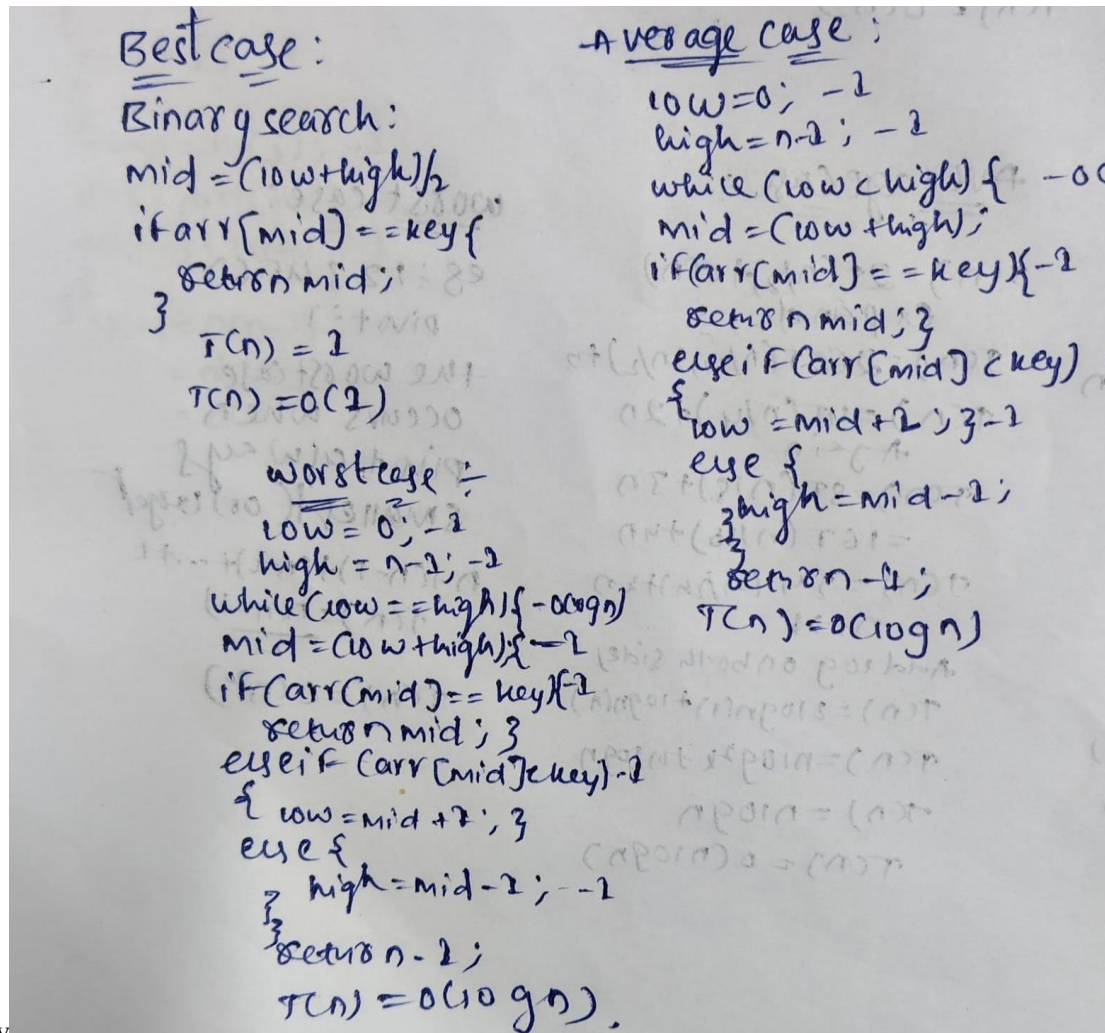


Output

```
Enter the number of elements: 4
Enter 4 elements in sorted order:
12
36
48
52
Enter the element to search for: 52
Element found at index: 3
```


+

Time complexity:



Best case:
Binary search:
 $mid = (low + high) / 2$
if $arr[mid] == key$ {
 return mid;
}
 $T(n) = 1$
 $T(n) = O(1)$

Worst case:
 $low = 0; -1$
 $high = n-1; -1$
while ($low \leq high$) {
 $mid = (low + high) / 2$
 if ($arr[mid] == key$) {
 return mid;
 }
 else if ($arr[mid] < key$) {
 $low = mid + 1; +1$
 }
 else {
 $high = mid - 1; -1$
 }
}
return -1;
 $T(n) = O(\log n)$

Average case:
 $low = 0; -1$
 $high = n-1; -1$
while ($low < high$) {
 $mid = (low + high) / 2$
 if ($arr[mid] == key$) {
 return mid;
 }
 else if ($arr[mid] < key$) {
 $low = mid + 1; +1$
 }
 else {
 $high = mid - 1; -1$
 }
}
return -1;
 $T(n) = O(\log n)$

Conclusion:

From this practical of the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.